МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

МЕТОДИЧНІ ВКАЗІВКИ

до виконання лабораторних робіт з дисциплін

«Сучасні Web-технології», та «Frontend-розробка»

для спеціальностей: F3 – «Комп'ютерні науки» та F1 – «Прикладна математика»

Лабораторна робота 3

Семантична розмітка HTML5

Харків

НТУ «ХПІ»

2026

ЗМІСТ

ЗМІСТ 2
ВСТУП 3
Мета роботи 3
Основні задачі: 3
РОЗДІЛ 1. ТЕОРЕТИЧНІ ВІДОМОСТІ 4
1.1. Поняття семантичної верстки 4
1.2. Основні семантичні елементи HTML5 5
1.3. Додаткові семантичні теги 8
1.4. Структурування веб-сторінки 9
РОЗДІЛ 2. ПРИКЛАД ВИРІШЕННЯ ЗАВДАННЯ 11
2.1. Постановка задачі 11
2.2. Покроковий алгоритм виконання 11
Завдання для самостійної роботи 17
Правила оформлення звіту про виконання лабораторної роботи 18
Контрольні питання 19
СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ 20

ВСТУП

Сучасний веб-розробник повинен володіти не лише навичками написання синтаксично правильного HTML-коду, але й розуміти принципи семантичної верстки, яка є фундаментом якісних веб-сайтів. Семантична розмітка передбачає використання HTML-елементів, які описують зміст та структуру сторінки, а не лише її візуальне оформлення. Такий підхід забезпечує краще сприйняття контенту пошуковими системами, покращує доступність для користувачів з обмеженими можливостями та спрощує підтримку та розвиток проекту в майбутньому.

Семантичні елементи HTML5 були введені для вирішення проблеми неструктурованого коду, який домінував у ранніх версіях мови розмітки. Традиційна верстка з використанням великої кількості елементів ускладнювала розуміння структури документа як людинами, так і автоматизованими системами. Нові семантичні теги, такі як <header>, <nav>, <main>, <article>, <section>, <aside>, <footer>, надають чітке значення кожній частині сторінки, що робить код більш зрозумілим та структурованим.

Мета роботи

Метою лабораторної роботи є вивчення семантичної структури HTML5-документа, освоєння основних семантичних елементів та отримання практичних навичок зі створення структурованих веб-сторінок з використанням семантичної розмітки.

Основні задачі:

1. Вивчити поняття семантичної верстки та її переваги.
2. Освоїти основні семантичні елементи HTML5.
3. Розглянути додаткові семантичні теги для форматування тексту.
4. Навчитися структурувати веб-сторінку за допомогою семантичних елементів.
5. Створити багатосторінковий веб-сайт з використанням семантичної розмітки.

6. Результати лабораторної роботи оформити у вигляді звіту зі скріншотами проміжних етапів.

РОЗДІЛ 1. ТЕОРЕТИЧНІ ВІДОМОСТІ

1.1. Поняття семантичної верстки

Семантична верстка — це підхід до створення веб-сторінок, при якому HTML-елементи використовуються відповідно до їх семантичного значення, а не лише для візуального оформлення. Семантика у програмуванні означає присвоєння значення коду або його частинам, і в контексті HTML це означає використання тегів за їх призначенням:

- заголовки позначаються тегами <h1>-<h6>,
- параграфи — тегом <p>,
- навігація — тегом <nav>,
- основний контент — тегом <main>.

Основна проблема традиційної верстки полягала у надмірному використанні тегу для створення будь-яких структурних блоків. Хоча є універсальним контейнером, він не несе жодного семантичного значення про свій вміст. Пошукові системи та допоміжні технології, такі як екранні читалки, не можуть визначити призначення кожного блоку без додаткових підказок. Семантичні елементи вирішують цю проблему, надаючи чітке значення кожній частині документа.

Переваги семантичної верстки є багатогранними та впливають на різні аспекти веб-розробки. По-перше, **пошукова оптимізація (SEO)** суттєво покращується, оскільки пошукові системи краще розуміють структуру та ієрархію контенту на сторінці. Семантичні елементи допомагають алгоритмам визначити, який контент є найважливішим, а який — допоміжним. По-друге, **доступність** для людей з обмеженими можливостями значно покращується завдяки екранним читалкам, які можуть коректно інтерпретувати структуру документа. По-третє, **підтримка коду** стає значно простішою, оскільки семантична розмітка робить код більш читабельним та зрозумілим для розробників.

Нижче наведено приклад коду **без** використання семантичних тегів та з ними.

```
<!-- Не семантичний підхід -->


<div class="header">
    <div class="logo">Мій Сайт</div>
    <div class="menu">
        <a href="#">Головна</a>
        <a href="#">Про нас</a>
    </div>
</div>

<!-- Семантичний підхід -->
<header>
    <div class="logo">Мій Сайт</div>
    <nav>
        <a href="#">Головна</a>
        <a href="#">Про нас</a>
    </nav>
</header>
```

1.2. Основні семантичні елементи HTML5

Стандарт HTML5 визначає низку семантичних елементів, які призначені для структурування веб-сторінки. Ці елементи поділяються на категорії залежно від їх призначення: елементи для структурування розділів сторінки та елементи для форматування текстового контенту.

Елементи для структурування розділів є основою семантичної верстки і визначають головні частини веб-сторінки.

Тег <header> позначає вступний контент або набір навігаційних посилань і може використовуватися як для всієї сторінки, так і для окремих секцій чи статей.

Тег <nav> призначений виключно для навігаційних посилань і позначає розділ сторінки з навігацією.

Тег <main> визначає основний контент сторінки і повинен бути унікальним для кожної сторінки.

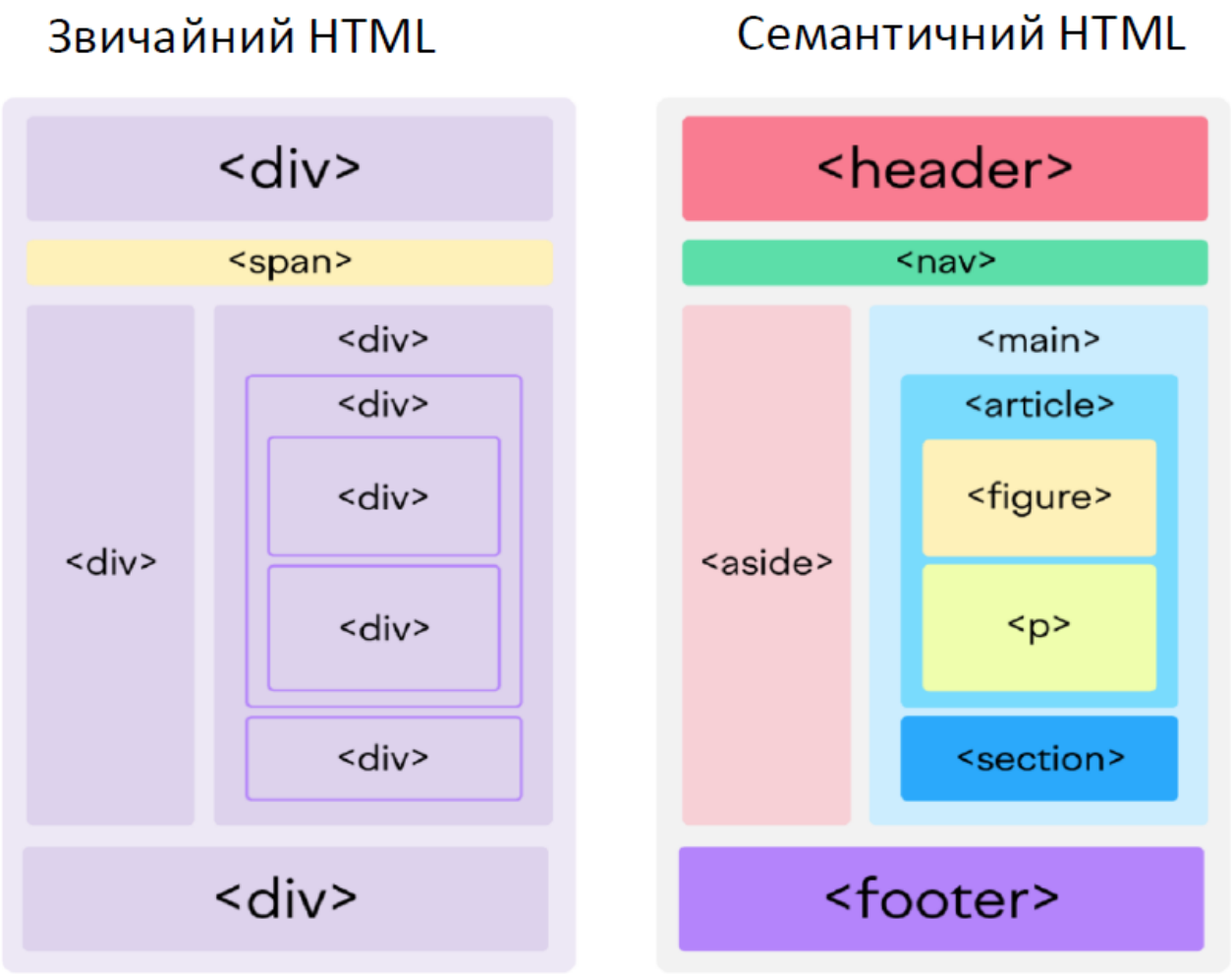

Рисунок 1. Звичайний HTML vs Семантичний

```
<header>
    <h1>Назва сайту</h1>
    <nav>
        <ul>
            <li><a href="/">Головна</a></li>
            <li><a href="/about">Про нас</a></li>
            <li><a href="/contact">Контакти</a></li>
        </ul>
    </nav>
</header>

<main>
    <article>
        <h2>Заголовок статті</h2>
        <p>Текст статті...</p>
    </article>
```

```
    <aside>
        <h3>Додаткова інформація</h3>
        <p>Бічний контент...</p>
    </aside>
</main>

<footer>
    <p>&copy; 2026 Мій Сайт. Всі права захищені.</p>
</footer>
```

Тег <article> позначає незалежний, самодостатній контент, який може існувати окремо від решти сторінки, наприклад стаття блогу, новина чи окремий пост.

Тег <section> використовується для логічного групування контенту всередині документа і зазвичай має заголовок.

Тег <aside> визначає контент, який побічно пов’язаний з основним контентом сторінки, наприклад бічну панель.

Тег <footer> позначає нижній колонтитул сторінки або розділу і зазвичай містить інформацію про автора, copyright чи посилання.

```
<!-- Приклад структури блогу -->
<article>
    <header>
        <h1>Заголовок статті</h1>
        <time datetime="2026-02-23">23 лютого 2026</time>
    </header>

    <section>
        <h2>Вступ</h2>
        <p>Текст вступу...</p>
    </section>

    <section>
        <h2>Основна частина</h2>
        <p>Текст основної частини...</p>
    </section>

    <footer>
        <address>Автор: Петренко Іван</address>
    </footer>
</article>
```

1.3. Додаткові семантичні теги

Окрім основних структурних елементів, HTML5 містить низку семантичних тегів для форматування текстового контенту, які надають браузерам та пошуковим системам додаткову інформацію про природу контенту.

Тег <figure> використовується для групування медіа-контенту, такого як зображення, діаграми чи ілюстрації, разом з підписом.

Тег <figcaption> визначає підпис для елемента <figure>.

Тег <mark> позначає виділений текст, який є релевантним у певному контексті, наприклад результати пошуку.

Тег <time> позначає час або дату і може використовуватися з атрибутом datetime для машинночитабельного формату.

```
<figure>
    <img src="diagram.png" alt="Діаграма структури HTML5">
    <figcaption>Рисунок 1 – Структура семантичних елементів HTML5</figcaption>
</figure>

<p>Конференція відбудеться <time datetime="2026-03-15">15 березня
2026</time>.</p>

<p>Важливе <mark>виділене</mark> слово в тексті.</p>
```

Тег <details> створює розкриваючий елемент, який приховає або показує додаткову інформацію за бажанням користувача.

Тег <summary> визначає видимий заголовок для елемента <details>.

Тег <address> позначає контактну інформацію для автора статті або власника сайту.

Тег <cite> використовується для посилання на джерело цитати.

Тег <code> позначає фрагмент програмного коду, а тег <var> — змінну в програмному коді.

```
<details>
    <summary>Додаткова інформація</summary>
    <p>Ця інформація прихована за замовчуванням.</p>
</details>

<address>
```

```
    Контакти автора:<br>
    Email: author@example.com<br>
    Телефон: +380 12 345 67 89
</address>

<p>Як зазначено у <cite>Документації MDN</cite>:</p>
<pre><code>const greeting = "Hello, World!";</code></pre>
```

1.4. Структурування веб-сторінки

Правильне структурування веб-сторінки передбачає ієрархічне розташування семантичних елементів, де кожен елемент має своє місце та призначення. Типова структура сторінки починається з елемента <header>, який містить логотип, назву сайту та навігацію. Далі йде елемент <main>, який охоплює основний контент сторінки. Всередині <main> можуть бути елементи <article> або <section> для окремих частин контенту. Бічні колонки або додаткова інформація розміщуються в елементі <aside>. Нижня частина сторінки позначається елементом <footer>.

```
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Семантична верстка HTML5">
    <title>Семантична розмітка HTML5</title>
</head>
<body>
    <header>
        <div class="logo">Логотип</div>
        <nav>
            <a href="#home">Головна</a>
            <a href="#news">Новини</a>
            <a href="#contact">Контакти</a>
        </nav>
    </header>

    <div class="container">
        <main>
            <article>
                <header>
                    <h1>Заголовок статті</h1>
                    <p>Опубліковано: <time datetime="2026-02-
23">23.02.2026</time></p>
                </header>
                <p>Вступний текст...</p>
```

```
                <section>
                    <h2>Підзаголовок розділу</h2>
                    <p>Текст розділу...</p>
                </section>

                <section>
                    <h2>Інший розділ</h2>
                    <p>Текст іншого розділу...</p>
                </section>

                <footer>
                    <address>Автор: Иван Петренко</address>
                </footer>
            </article>
        </main>

        <aside>
            <h2>Бічна панель</h2>
            <nav>
                <h3>Навігація</h3>
                <ul>
                    <li><a href="#">Посилання 1</a></li>
                    <li><a href="#">Посилання 2</a></li>
                </ul>
            </nav>
        </aside>
    </div>

    <footer>
        <p>&copy; 2026. Усі права захищені.</p>
    </footer>
</body>
</html>
```

При структуруванні сторінки важливо дотримуватися принципу логічної ієрархії та використовувати елементи за їх призначенням. Елемент <header> не обов’язково повинен бути на початку сторінки — він може використовуватися всередині статті чи секції. Елемент <section> слід використовувати для групування пов’язаного контенту, який має заголовок. Елемент <article> призначений для контенту, який може бути розповсюджений незалежно від іншої частини сторінки.

РОЗДІЛ 2. ПРИКЛАД ВИРІШЕННЯ ЗАВДАННЯ

2.1. Постановка задачі

У рамках лабораторної роботи необхідно переробити веб-сайт, створений у попередній лабораторній роботі, з використанням семантичної розмітки HTML5. Завдання включає створення нової сторінки для блогу або новин, структурування існуючих сторінок за допомогою семантичних елементів та застосування якомога більшої кількості різних семантичних тегів для демонстрації їх призначення.

2.2. Покроковий алгоритм виконання

Крок 1. Підготовка проекту

Відкрийте проект, створений у попередній лабораторній роботі, в CodeSandbox або VS Code. Переконайтеся, що структура директорій містить необхідні папки. Оновіть локальний репозиторій командою git pull, якщо працюєте з віддаленим репозиторієм. Створіть нову гілку для роботи над семантичною версткою командою

```
git checkout -b lab3-semantic
```

Крок 2. Аналіз існуючого коду

Відкрийте файл index.html та проаналізуйте його структуру. Визначте, які частини сторінки можна замінити на семантичні елементи. Зазвичай замінюється на <header>, або на <nav>, або на <main>, а на <footer>.

Крок 3. Переробка головної сторінки

Відкрийте файл index.html і замініть існуючу структуру на семантичну. Додайте елемент <header> з навігацією <nav>. Оберніть основний контент в елемент <main>. Додайте елемент <footer> з інформацією про сайт. Використайте тег <nav> для навігаційного меню замість звичайних списків.

```html
<!-- Оновлена структура index.html -->
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Головна сторінка</title>
</head>
<body>
    <header>
        <h1>Назва сайту</h1>
        <nav>
            <ul>
                <li><a href="index.html">Головна</a></li>
                <li><a href="pages/about.html">Про нас</a></li>
                <li><a href="pages/blog.html">Блог</a></li>
                <li><a href="pages/contact.html">Контакти</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <section class="hero">
            <h2>Ласкаво просимо</h2>
            <p>Текст вітального повідомлення...</p>
        </section>

        <section class="features">
            <h2>Наші переваги</h2>
            <article>
                <h3>Перевага 1</h3>
                <p>Опис переваги...</p>
            </article>
            <article>
                <h3>Перевага 2</h3>
                <p>Опис переваги...</p>
            </article>
        </section>
    </main>

    <footer>
        <p>&copy; 2026 Назва сайту. Всі права захищені.</p>
    </footer>
</body>
</html>
```

Крок 4. Створення сторінки блогу

Створіть новий файл pages/blog.html для сторінки блогу. Використайте повну семантичну структуру з елементами <article> для кожного посту, <time> для дат публікації, <figure> та <figcaption> для зображень з підписами.

```
<!-- pages/blog.html -->
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Блог</title>
</head>
<body>
    <header>
        <h1>Мій блог</h1>
        <nav>
            <ul>
                <li><a href="../index.html">Головна</a></li>
                <li><a href="blog.html">Блог</a></li>
                <li><a href="contact.html">Контакти</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <article>
            <header>
                <h2>Перша стаття</h2>
                <p>Опубліковано: <time datetime="2026-02-20">20 лютого
2026</time></p>
            </header>
            <p>Вступ до статті...</p>

            <section>
                <h3>Підзаголовок</h3>
                <p>Текст розділу...</p>
            </section>

            <figure>
                <img src="../images/article1.jpg" alt="Зображення до статті">
                <figcaption>Рисунок 1 — Ілюстрація до статті</figcaption>
            </figure>

            <footer>
                <address>Автор: <a href="mailto:author@example.com">Іван
```

```
Петренко</a></address>
            </footer>
        </article>

        <article>
            <header>
                <h2>Друга стаття</h2>
                <p>Опубліковано: <time datetime="2026-02-15">15 лютого
2026</time></p>
            </header>
            <p>Вступ до другої статті...</p>
        </article>
    </main>

    <footer>
        <address>
            <p>Контакти: email@example.com</p>
        </address>
        <p>&copy; 2026 Мій блог</p>
    </footer>
</body>
</html>
```

Крок 5. Використання додаткових семантичних тегів

Оновіть існуючі сторінки з використанням додаткових семантичних тегів. Додайте на сторінки теги <details> та <summary> для розкриваючих секцій, <mark> для виділення важливого тексту, <code> та <pre> для відображення програмного коду, <address> для контактної інформації.

```
<!-- Приклад використання додаткових тегів -->
<details>
    <summary>Детальніша інформація</summary>
    <p>Ця інформація прихована і розкривається за бажанням користувача.</p>
</details>

<p>Для виведення тексту використовуйте <code>console.log()</code>.</p>

<pre><code>function greet(name) {
    return `Hello, ${name}!`;
}</code></pre>

<mark>Важливе зауваження</mark> в тексті.
```

Крок 6. Створення сторінки контактів

Створіть файл pages/contact.html з повною семантичною структурою. Використайте елемент <address> для контактної інформації, <form> з відповідними семантичними елементами для форми зворотного зв’язку.

```
<!-- pages/contact.html -->
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Контакти</title>
</head>
<body>
    <header>
        <h1>Контакти</h1>
        <nav>
            <ul>
                <li><a href="../index.html">Головна</a></li>
                <li><a href="blog.html">Блог</a></li>
                <li><a href="contact.html">Контакти</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <section>
            <h2>Наші контакти</h2>
            <address>
                <p><strong>Адреса:</strong> м. Харків, вул. Прикладна, 1</p>
                <p><strong>Телефон:</strong> <a href="tel:+380123456789">+380 12
345 67 89</a></p>
                <p><strong>Email:</strong> <a
href="mailto:info@example.com">info@example.com</a></p>
            </address>
        </section>

        <section>
            <h2>Форма зворотного зв'язку</h2>
            <form action="#" method="post">
                <p>
                    <label for="name">Ім'я:</label>
                    <input type="text" id="name" name="name" required>
                </p>
                <p>
                    <label for="email">Email:</label>
                    <input type="email" id="email" name="email" required>
                </p>
```

```
                <p>
                    <label for="message">Повідомлення:</label>
                    <textarea id="message" name="message" rows="5"
required></textarea>
                </p>
                <button type="submit">Надіслати</button>
            </form>
        </section>
    </main>

    <footer>
        <p>&copy; 2026. Всі права захищені.</p>
    </footer>
</body>
</html>
```

Крок 7. Перевірка структури

Перегляньте всі сторінки в браузері. Переконайтеся, що семантичні елементи не змінили візуальне відображення контенту. Використайте Chrome DevTools для перевірки структури документа. Переконайтеся, що кожна сторінка має логічну ієрархію заголовків від <h1> до <h6>.

Крок 8. Збереження та публікація

Збережіть всі зміни у файлах проекту. Зробіть коміт змін командою

```
git add .
```

та

```
git commit -m "Add semantic HTML5 structure"
```

Відправте зміни на GitHub командою

```
git push origin lab3-semantic
```

Опублікуйте проект на CodeSandbox або переконайтеся, що GitHub Pages оновлено.

Крок 9. Фіналізація

Переконайтеся, що всі сторінки доступні та працюють коректно. Підготуйте скріншоти для звіту. Збережіть посилання на репозиторій та опублікований сайт.

Завдання для самостійної роботи

Додатково виконайте і додайте до звіту щонайменше 2 приклади відповідно до наведених нижче варіантів.

Номер варіанту	Номер завдання з переліку нижче	
Варіант 1	1	7
Варіант 2	2	8
Варіант 3	3	9
Варіант 4	4	10
Варіант 5	5	11
Варіант 6	6	12

1. **Семантизація сторінки «Про нас».** Переробіть сторінку, створену в попередній лабораторній роботі, з використанням семантичних елементів <header>, <main>, <section>, <article>, <footer>.

2. **Створення сторінки новин.** Створіть сторінку з новинами інституту. Використайте елементи <article> для кожної новини та <time> для дат публікації.

3. **Галерея з підписами.** Створіть сторінку з галереєю зображень. Використайте теги <figure> та <figcaption> для кожного зображення з підписом.

4. **FAQ-сторінка.** Створіть сторінку з частими питаннями. Використайте теги <details> та <summary> для створення розкриваючих секцій.

5. **Сторінка команди.** Створіть сторінку з інформацією про викладачів. Використайте <article> для карток кожного викладача з фотографією та контактною інформацією.

6. **Сторінка з кодом.** Створіть сторінку з прикладами програмного коду. Використайте теги <code>, <pre>, <var> для форматування коду.

7. **Сторінка подій.** Створіть сторінку з календарем подій. Використайте тег <time> з атрибутом datetime для кожної дати.

8. **Блог з категоріями.** Розширте сторінку блогу, додавши категорії для статей. Використайте <aside> для бічної панелі з навігацією по категоріях.

9. **Сторінка документації.** Створіть сторінку з технічною документацією. Використайте <details> для розкриваються секцій та <code> для прикладів коду.

10. **Портфоліо.** Створіть сторінку портфоліо з проектами. Кожен проект оформіть як окремий <article> з зображеннями та описом.

11. **Резюме.** Створіть сторінку з резюме студента. Використайте семантичні теги для структурування освіти, досвіду роботи, навичок.

12. **Дослідження тегів.** Використайте п’ять семантичних тегів, які не були розглянуті в прикладі: <mark>, <output>, <progress>, <meter>, <wbr>. Опишіть їх призначення та продемонструйте використання.

Правила оформлення звіту про виконання лабораторної роботи

Звіт про виконання лабораторної роботи повинен містити:

1. Титульний аркуш.

2. Мету роботи.

3. Результати виконання роботи:

   - виконане перше покрокове завдання, що складається з покрокового налаштування середовищ розробки зі скрінами проміжних етапів;

   - виконане одне з наведених вище варіантів зі скрінами проміжних етапів;

4. Розгорнуті висновки по виконаній роботі з зазначенням можливих складнощів.

Всі копії екранів повинні бути оформлені як рисунки з відповідними підписами під ними (наприклад: Рисунок 1.1 – Копія екрану з результатами роботи програми). Рисунки слід оформляти на зразок тих, що наведені в даній лабораторній роботі. На кожен з рисунків повинно бути посилання в тексті звіту.

Контрольні питання

1. Що таке семантична верстка і чому вона важлива?
2. Які переваги надає використання семантичних елементів для SEO?
3. Чим відрізняється тег <section> від тегу <article>?
4. Які елементи використовуються для створення навігаційного меню?
5. Для чого використовується тег <aside> і де він зазвичай розміщується?
6. Яка різниця між тегами <header> та <h1>?
7. Які семантичні теги використовуються для форматування текстового контенту?
8. Для чого використовується атрибут datetime у тезі <time>?
9. Як правильно використовувати теги <figure> та <figcaption>?
10. Чому важливо дотримуватися ієрархії заголовків від <h1> до <h6>?
11. Як тег <details> покращує користувацький досвід?
12. Для чого використовується тег <address> і де він може бути розміщений?
13. Яка різниця між тегами та <section>?
14. Як екранні читалки використовують семантичну розмітку?
15. Скільки разів на сторінці може використовуватися елемент <main>?

16. Для чого використовується тег <nav> і чи обов’язково він повинен бути один?

17. Як тег <mark> покращує сприйняття контенту?

18. Чому тег <article> вважається самодостатнім елементом?

19. Які теги використовуються для відображення програмного коду?

20. Як правильно структурувати сторінку з кількома статтями?

21. Для чого використовується тег <footer> всередині елемента <article>?

22. Чому семантична верстка полегшує підтримку коду?

23. Які атрибути можна використовувати з тегом <time>?

24. Як тег <cite> використовується для оформлення цитат?

25. Чим відрізняється візуальне відображення від <header>?

СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ

1. Бородкіна І. Л. Web-технології та web-дизайн: застосування мови HTML для створення електронних ресурсів : навч. посіб. / І. Л. Бородкіна, Г. О. Бородкін. – Київ : Ліра-К, 2020. – 210 с.

2. MDN Web Docs. HTML: HyperText Markup Language [Електронний ресурс]. – Режим доступу: https://developer.mozilla.org/uk/docs/Web/HTML

3. HTML Living Standard [Електронний ресурс]. – Режим доступу: https://html.spec.whatwg.org/

4. Semantic HTML [Електронний ресурс]. – Режим доступу: https://developer.mozilla.org/uk/docs/Web/HTML/Content_categories

5. HTML Elements Reference [Електронний ресурс]. – Режим доступу: https://developer.mozilla.org/uk/docs/Web/HTML/Element

6. W3Schools HTML5 Tutorial [Електронний ресурс]. – Режим доступу: https://www.w3schools.com/html/

7. Доступність веб-контенту (WCAG) [Електронний ресурс]. – Режим доступу: https://www.w3.org/WAI/WCAG21/quickref/

8. Web Accessibility Initiative (WAI) [Електронний ресурс]. – Режим доступу: https://www.w3.org/WAI/

9. SEO Guide for Web Developers [Електронний ресурс]. – Режим доступу: https://developer.mozilla.org/en-US/docs/Glossary/SEO

10. HTML5 Semantic Elements Examples [Електронний ресурс]. – Режим доступу: https://www.w3schools.com/html/html5_semantic_elements.asp